AFP-Enterprise: Evaluating Host-Level Runtime Containment for Planner–Tool Execution

Abstract

Enterprise agent stacks can plan and can move messages. They still lack a runtime answer at the execution boundary: *may this planner or tool step run?*

This report specifies and records **AFP-Enterprise**: a host runtime kernel beside a local planner, reached by a synchronous pre-flight probe. Before irreversible tool I/O, the planner reports recursion depth and context, then blocks on allow, delay, or isolate. The kernel does not parse prompts or tool schemas. It checks, in order, an emergency stop, a depth cap, a four-axis pressure maximum, and a specified isolate–recover state machine.

On a personal computer, a LangGraph planner issued real tool I/O (local lookup and loopback HTTP GET). Five bounded jobs completed under no gate, a per-step wipe, and the current sidecar (false isolate = 0). A scripted infinite loop and a 48-task burst ran to completion without a gate and were contained by both gated variants. Those traces fire the depth cap and the hard pressure isolate; they do not reach the delay band or the recover machine. The gated variants differ only when a later probe reuses leftover host telemetry: AFP-Enterprise refuses; the wipe allows. A new job that reported depth 0 was admitted under both.

The measured increment is **retained execution telemetry** (stale-state retention), not an identity-persistent penalty, production safety, LLM-agent validation, or general prevention of resource exhaustion. A 500-node Monte Carlo in the appendix is a breaker invariant, not a field result.

Keywords: multi-agent systems, planner–tool runtime, admission control, pre-flight check, retained telemetry, stale-state retention, circuit breaker

---

Read this first: names

This report uses full names in prose. Short forms appear in code and older documents.

Full name	Short form	Meaning in this report
AFP-Enterprise	—	Host runtime kernel for a local planner and its tools. Pre-flight is the call, not the whole kernel.
Pre-flight check	PreFlight	Blocking RPC: may this step run?

Full name	Short form	Meaning in this report
Host kernel	—	Ordered stages: emergency stop, depth cap, pressure isolate, isolate–recover machine.
System pressure	entropy / `entropy_load`	A 0–1 maximum of concurrency, memory, context, and burst.
Retained execution telemetry	leftover depth	Last-reported host metrics kept until a later report overwrites them.

What this report does *not* treat as measured: Coordination Viability Probability, stranger tax, gossip, governance headers, or a lock on agent identity. Those appear in older mixed drafts. They are not evidence in this report.

Table of contents

1. The empty cell
2. What AFP-Enterprise is and is not
3. Mechanism (interface, four-stage kernel, pressure axes, delay, isolate–recover math)
4. Specified persistence versus shipped retention
5. Implementation sketch
6. Empirical record
7. Evidence matrix and reproduction
8. Related work
9. What this report does not claim
10. Closing

---

1. The empty cell

1.1 Plan, transport, no runtime law

Multi-agent infrastructure has matured along two axes:

1. **Plan** — frameworks such as LangGraph, CrewAI, and AutoGen decompose work, delegate, and replan.
2. **Move messages** — gRPC, HTTP, Kafka, and MCP deliver bytes.

A third cell is empty. Before each coordination step commits, something must answer:

> May this step execute?

Layer	Representative tools	What is missing
Planning	LangGraph, CrewAI, AutoGen	Decomposition without a physical brake
Transport	gRPC, HTTP, Kafka, MCP	Bytes arrive; an internal task storm may never touch the wire
Semantic collaboration	MCP, A2A, session protocols	Negotiate *what* may be said — traffic lights, not brakes
Coordination runtime	**AFP-Enterprise (this report)**	Host kernel before a local tool hop; record what the host last reported

AFP-Enterprise does not compete with transports or workflow engines. It occupies the gap between *agents can talk* and *this host still survives*. Resource admission is decided by physical constraints at the execution boundary. There is no bidding, auction, or chip ledger.

One language-model planner plus MCP or local tools is already enough to fail. Multi-planner conversation is not required.

1.2 Why stacking is the failure geometry

The dominant failure geometry is **agent stacking**: planner → tool → planner → …, sometimes closing a loop.

Each hop can be a valid tool call. The *trajectory* can still become physically unsustainable: recursion depth, context volume, concurrency, and burst grow until the node saturates.

Per-request gateways forget. Token budgets often account after commit. Framework `max_retries` and `recursion_limit` reset when the session ends. In-process guardrails share the optimizer's address space.

Pattern	What it helps	Structural gap
Retry / loop counters	Cap iterations inside one graph	Reset when the session ends; one "retry" can hide fan-out
Token / cost budgets	Stop after spend	Post-commit; internal queues can swell with no wire traffic
Metrics and alerts	Discover incidents	Often after damage; alerts are not gates
In-process guardrails	Developer-declared caps	Bypassable, not a separate process

```
Observability path:     execute → count tokens → alert → stop this episode
AFP-Enterprise:        report state → kernel (stop / depth / pressure / recover)
→ allow / delay / isolate → then I/O
```

1.3 Three structural pathologies

These are default optimizer behaviors, not rare bugs.

Intent burst. One objective fractures into sub-objectives; each spawns tool chains. Wire metrics can stay quiet while an in-process queue grows. Example: “summarize risk for every name in the index” fans into filings, ratios, and peer comps until memory pressure trips the node.

Recursive delegation loop. Expressive control flow needs cycles: replan, reflect, retry. The session can stay syntactically valid while depth climbs. Example: a debugger that re-delegates “find the root cause” until the process burns CPU with little outbound I/O.

Context avalanche. Bounded depth does not bound state volume. Each step appends passages and transcripts; cost grows even if the recursion cap is not hit.

AFP-Enterprise adjudicates the physical question — will this step blow through depth, pressure, or burst? — not “was this a hallucination?” and not “is the business answer correct?”

1.4 The scene this report covers

The intended deployment is a **local planner plus MCP or local tools**, on hosts the operator already controls.

This report does not cover untrusted inbound traffic from other organizations.

A synthetic 500-node mesh from an earlier draft is **not** a result about that planner–tool scene. If it appears at all, it is an appendix invariant of a toy breaker (Section 6.7).

---

2. What AFP-Enterprise is and is not

2.1 One sentence

Legal planner steps and tool calls can still stack a machine. AFP-Enterprise is a **host kernel** that answers, before the tool runs, whether this step may proceed. Pre-flight is how the planner calls that kernel. Bounded jobs that were tested can complete. Tested loops and bursts can be stopped by the depth cap or the hard pressure isolate. What is remembered afterwards is last-reported host telemetry until a later report overwrites it — **not** a lock on agent identity.

2.2 Planner interface

The planner does not embed the kernel. Before a tool call it writes telemetry, then blocks on `PreFlightCheck`. The sidecar does not parse prompts or tool schemas. Each hop is write-then-probe:

1. report recursion depth and context bytes;
2. probe with a burst hint (estimated imminent fan-out);
3. proceed only on allow; sleep on delay; halt I/O on isolate.

```
Planner / tools
      │  write-then-probe
      ▼
Host kernel
  emergency stop · depth cap · four-axis pressure · isolate-recover machine
      │
      ▼
allow  /  delay(δ)  /  isolate
```

The probe must finish before irreversible intent or I/O. The planner must not continue until the verdict returns. There is no advisory-only mode.

The reference placement is an out-of-process **sidecar** (Unix-domain socket and gRPC). A Python SDK wraps LangGraph nodes and tools so a blocked step annotates graph state and stops, instead of throwing the process into an ungoverned retry storm.

2.3 What it is not

Not	Why
Alignment or injection defense	Does not replace AgentDojo, AI control, or MCP security
Task correctness	Does not grade answers
Network ingress filter	Does not police traffic from other organizations
Validated field safety	Laptop-controlled runs, not a production fleet
Identity-locked persistence	Specified in older drafts; **the shipped sidecar keeps last-reported depth**
A fully measured recover path	Delay band and isolate–recover machine are specified; these traces do not reach them

---

3. Mechanism

3.1 One kernel per sidecar

Each sidecar hosts one kernel. Every pre-flight probe from the local planner must ask that same object.

In the **current implementation**, an optional destination field on the probe is **ignored**. State is kept on the host sidecar, not keyed to one agent identity.

3.2 Timing contract

An action is pre-intent if and only if it is evaluated at the execution boundary before either:

1. the planner commits to a new externally visible intent (tool call, delegation, outbound message), or
2. the runtime performs irreversible cross-boundary I/O.

Parsing RPC after the fact is observation, not this kernel. Relative to the planner's decision, pre-flight is **synchronous**.

The check does not parse prompts, tool schemas, or session text. The contested resource is **optimization capacity**.

3.3 Write-then-probe

Each planner hop and each tool call:

1. `ReportInternalState(recursion_depth, context_bytes)`
2. `PreFlightCheck(estimated_tasks, trace_id)`
3. Allow → execute. Isolate → annotate graph state and stop. Do not convert isolate into an ungoverned retry storm.

```
PreFlightRequest {
  trace_id        : string
  estimated_tasks : uint32    // burst hint
}

PreFlightResponse {
  action       : PERMISSIVE | THROTTLED | ISOLATED
  delay_ms     : uint32
```

```
  block_reason : string
}
```

Action	Optimizer obligation
Allow (`PERMISSIVE`)	Proceed
Delay (`THROTTLED`)	Sleep at least `delay_ms`; re-probe if policy requires
Isolate (`ISOLATED`)	Halt I/O; `block_reason` names the circuit

If the internal-state report is omitted or stale, pressure at probe time is **under-measured**. The protocol assumes the runtime reports depth and context; OS-level memory signals bound some under-reporting.

3.4 Host kernel (four stages)

The kernel evaluates four stages in this order and returns on the first match.

1. **Emergency stop.** An operator kill switch isolates every hop, regardless of measured pressure.
2. **Depth cap.** Reported depth (> \texttt{MaxRecursionDepth}) (default (10)) isolates immediately. Pressure need not already sit in the warn band.
3. **Pressure isolate.** If (e \ge e^{\star}) (default (0.95)), isolate **before** the isolate–recover machine is consulted.
4. **Isolate–recover machine.** Only if the first three stages pass.

These traces fire stage 2 (the loop) or stage 3 (the burst). They do not trip the emergency stop, the delay band, or the recover machine.

3.5 How system pressure is computed

System pressure (e \in [0,1]) is a scalar at the execution boundary. It is a **maximum**, not an average. If any axis saturates, pressure saturates.

[e \;=\; \max!\bigl(e_{\mathrm{tool}},\, e_{\mathrm{mem}},\, e_{\mathrm{ctx}},\, e_{\mathrm{burst}}\bigr)]

Axis	Formula (shipped)	Default	What it captures
Tool	(e_{\mathrm{tool}} = \min!\bigl(1,\; n_{\mathrm{active}} / C\bigr))	(C = 50)	In-flight tool calls

Axis	Formula (shipped)	Default	What it captures
Memory	$e_{\mathrm{mem}} = \min!\bigl(1,\; \mathrm{mem} / r_{\mathrm{oom}}\bigr)$	$r_{\mathrm{oom}} = 0.90$	Substrate / OOM proximity
Context	$e_{\mathrm{ctx}} = \min!\bigl(1,\; b / B\bigr)$	$B = 512\,\mathrm{MiB}$	Reported planning bytes
Burst	$e_{\mathrm{burst}} = \min!\bigl(1,\; \hat{n} / C\bigr)$	$C = 50$	Prospective fan-out

n_{active} is the live tool-call counter. mem is the host memory-usage ratio. b is last-reported context bytes. $\hat{n}$ is `estimated_tasks` on the probe; if the hint is 0, the shipped monitor substitutes 1.

The storm job's hint $\hat{n}=48$ gives $e_{\mathrm{burst}} = 48/50 = 0.96$. That is why S4 isolates on stage 3 with zero GETs.

Band	Symbol	Value	Effect
Safe	e_{safe}	0.40	Machine may return to allow
Warn	e_{warn}	0.75	Delay path (stage 4)
Breaker	$e^{\star}$	0.95	Hard isolate (stage 3)

Depth is a separate discrete breaker, not folded into e:

```
reported_depth > MaxRecursionDepth  ⇒  ISOLATED     # default 10
```

That is the answer to a recursive delegation loop. S3 reports depth 11 and isolates here.

3.6 Delay injection (specified; not reached on these traces)

If stage 4 chooses delay, the shipped latency function is piecewise:

$$\delta(e) \;=\; \begin{cases} 500\,\mathrm{ms} & e \le e_{\mathrm{warn}} \\ 500 + 1500\cdot\dfrac{e - e_{\mathrm{warn}}}{1 - e_{\mathrm{warn}}}\,\mathrm{ms} & e > e_{\mathrm{warn}} \end{cases}$$

So delay is $500\,\mathrm{ms}$ at the warn line and $2000\,\mathrm{ms}$ as $e \to 1$. The planner must sleep at least δ before the next hop. A probation probe uses a fixed $1000\,\mathrm{ms}$ window instead of $\delta(e)$.

These traces never enter that path: S3 and S4 return at stages 2–3.

3.7 Isolate–recover machine (specified; not reached on these traces)

Stage 4 is a four-state machine: **allow**, **delay**, **isolate**, **probation**. Floor rules fire before state-specific logic:

```
invalid sign  ∨  malicious spike  ∨  recover_score < 0.3  ⇒  isolate
```

On this sidecar a malicious spike is $\hat{n} > C$ (burst hint above the concurrency cap). S4 uses $\hat{n}=48 < 50$, so the spike rule does not fire; the pressure breaker does.

State transitions (shipped constants):

Current	Condition	Next	Visible action
Allow	$e > 0.75$	Delay	sleep $\delta(e)$
Allow	otherwise	Allow	proceed
Delay	$e < 0.40$	Allow	proceed
Delay	otherwise	Delay	sleep $\delta(e)$
Isolate	epochs since entry ≥ 64	Probation	low-rate probe, $1000\,\mathrm{ms}$
Isolate	otherwise	Isolate	halt
Probation	$e > 0.40$	Isolate	halt (clock stamps on re-entry)
Probation	dwell (> 128) **and** recover score ≥ 0.8	Allow	proceed
Probation	otherwise	Probation	low-rate probe

Hysteresis: leaving delay requires $e < 0.40$, not merely $e < 0.75$. That dead band is to stop oscillation on the warn line.

The isolate clock stamps **on entry**. Later dropped probes must not refresh it; otherwise continuous traffic would starve recovery.

On the current local sidecar the recover score is **held at (1.0)** for the single host bucket. Formula A below is therefore idle on these traces. The machine is still in the binary; stages 2–3 return first.

3.8 Recover-score formulas (specified; idle on these traces)

Older drafts call this score Coordination Viability Probability. This report keeps the shipped equations and does not treat them as measured.

Formula A — evolution under evidence ($\alpha = 0.95$, $\beta = 0.05$):

$$v_{\mathrm{new}} \;=\; \mathrm{clamp}!\bigl(\alpha\, v_{\mathrm{old}} + \beta\, s - \gamma\, p - \delta\, e,\; 0,\; 1 \bigr)$$

with the shipped coefficients $\gamma = e^{p}-1$ and $\delta = 0.1\, e$. (s) is throughput success. (p) is a destabilization penalty.

Term	Meaning
αv_{old}	Historical inertia
βs	Reward cooperative completion
γp	Magnify a spike / invalid sign
δe	Couple physical pressure to the score

Formula B — idle decay ($\lambda = 0.01$) per epoch):

$$v_{\mathrm{eff}} \;=\; v_{\mathrm{hist}}\cdot e^{-\lambda\,\Delta t}$$

Formula C — logarithmic recovery (floor (0.3), $\kappa = 0.02$)):

$$v_{\mathrm{rec}}(\Delta t) \;=\; \min!\bigl(1,\; 0.3 + \kappa\cdot\log(1+\Delta t_{\mathrm{probation}})\bigr)$$

Hard floor: $v < 0.3$ isolates, same path as a malicious spike. Full return to allow also requires $v \ge 0.8$ after the (128)-epoch probation.

This laptop evaluation does not validate Formulas A–C or the dwells. Do not read Section 6 as a recover-score or identity-hysteresis result.

---

4. Specified persistence versus shipped retention

Older drafts said: the isolate verdict is stored on identity, survives the request, and clears only after asymmetric recovery.

The shipped sidecar is not that.

Depth isolation follows **last-reported recursion depth** on the host monitor. A new graph that reports `depth = 0` is admitted after isolate. A per-step wipe that clears depth at request start **matches** the current sidecar on the bounded and first-containment jobs. They differ only on a stale post-isolate probe that does not rewrite depth.

Claim	Holds now?
Pre-flight can stop the tested loop and burst	Yes, on those jobs
Tested bounded jobs are not false-isolated	Yes, on those jobs
Consequences stay on identity until recovery dwell	**No**
Two planners on one sidecar do not overwrite each other	**No** (the later report wins)

Calling the third or fourth row "validated" is an overclaim. The accurate object is **retained execution telemetry / stale-state retention**.

---

5. Implementation sketch

All measurements in this report used a local sidecar and the Python SDK. Kubernetes is a supported deployment shape; it is not the evaluation harness.

Piece	Role
Sidecar process	Hosts the kernel and IPC
Host kernel	Emergency stop, depth cap, pressure isolate, isolate–recover machine
Pressure monitor	Last-reported depth, context, concurrency, memory
Isolate–recover machine	Specified; not reached on these traces
Python SDK	Report-then-probe helpers
LangGraph wrappers	Node and tool wrappers that stop on isolate
Trace scripts	Deterministic planner plus real tools

Default policy (overridable): recursion cap 10, tool concurrency cap 50, breaker 0.95.

---

6. Empirical record

All runs were on one personal computer. The default planner is a **deterministic script** (reproducible in CI). That isolates sidecar admission from stochastic LLM routing; the live 0.5B arm is recorded separately in Section 6.6. Tools are real: a local key-value lookup and HTTP GET against a loopback fixture. The graph is identical across gates.

Evidence layers:

1. **Primary:** scripted planner + real tools + gate on versus off.

2. **Comparative:** no gate / per-step wipe / current sidecar.
3. **Supplementary:** local Qwen2.5 0.5B choosing tools.
4. **Appendix only:** 500-node Monte Carlo (breaker invariant).

6.1 Depth-only (no tools)

Source: `runtime_trace.csv` (1 October 2026).

Scene	Gate	Finished	Steps	Probes	Isolated	Depth	Wall (ms)
Benign	off	yes	3	0	no	3	2.0
Benign	on	yes	3	3	no	3	3.8
Empty spin	off	to cap	40	0	no	40	33.5
Empty spin	on	no	11	12	yes	11	25.9

This is a unit check before tools were wired. Table 6.2 is the main tool result.

6.2 Primary table: scripted planner, real tools

Source: `llm_tool_trace.csv` (1 October 2026). Scripted backend; 0 hosted tokens.

Job	Gate	Done	Tools	Probes	Isolated	Depth	Wall (ms)	Reason
S1 short (one lookup)	off	yes	1	0	no	2	1.0	—
S1 short	on	yes	1	3	no	2	4.6	—
S2 long (4 lookups + 4 GETs)	off	yes	8	0	no	9	7.8	—
S2 long	on	yes	8	17	no	9	8.6	—
S3 runaway `tick`	off	to cap	39	0	no	40	8.8	—
S3 runaway	on	no	10	22	yes	11	8.0	recursion cap
S4 storm `expand n=48`	off	yes	49	0	no	50	30.3	—
S4 storm	on	no	0	2	yes	1	1.7	burst entropy 0.96

False isolate on S1/S2: 0. Wall-clock is tens of milliseconds. That is not a load test.

Sentence that may be written:

> In a controlled planner–tool runtime, the bounded jobs that were tested can complete, and the tested loop and burst can be stopped by the gate.

Sentence that must not be written:

> AFP prevents multi-agent resource exhaustion.

6.3 Three-way baseline

Bounded jobs expanded to five: S1, S2, lookups only, fetches only, and eight repeats of the same lookup.

Off: no pre-flight.
Stateless / per-step: same sidecar checks, but each request starts by reporting depth 0.
Current sidecar: leftover depth is kept until overwritten.

Gate	Bounded done	S3/S4 isolated	Stale probe, no rewrite	New job reports 0
Off	5/5	0/2	—	—
Per-step wipe	5/5	2/2	allow	done
Current sidecar	5/5	2/2	refuse	done

False isolate on bounded jobs: 0. Mean wall-clock on the five bounded jobs in this run: **3.8 ms off, 6.4 ms wipe, 6.0 ms current sidecar**. About 2.2 ms extra versus off for the current sidecar. Indicative only. Not a throughput or scalability benchmark.

Scene	No gate	Per-step wipe	Current sidecar
Five bounded jobs	complete	complete	complete
False isolate	0	0	0
Infinite loop / 48-burst	not stopped	stopped	stopped
Stale probe, uncleared telemetry	—	allow	**refuse**
New job reports depth 0	—	allow	allow

Reading. S1–S4 do not distinguish the two gated variants. They differ only when a later execution reuses stale, uncleared telemetry. The increment is stale-state containment. Do not write "AFP-Enterprise provides consequence persistence."

Source: `baseline_trace.csv` (7 October 2026 — latest run). Every job row:

Job	Gate	Done	Tools	Probes	Isolated	Wall (ms)	Reason
S1 short	off	yes	1	0	no	1.6	—
S1 short	wipe	yes	1	3	no	2.0	—
S1 short	sidecar	yes	1	3	no	2.0	—
S2 long	off	yes	8	0	no	8.3	—
S2 long	wipe	yes	8	17	no	9.4	—
S2 long	sidecar	yes	8	17	no	8.7	—
Lookups only	off	yes	4	0	no	1.5	—
Lookups only	wipe	yes	4	9	no	3.8	—
Lookups only	sidecar	yes	4	9	no	4.2	—
Fetches only	off	yes	6	0	no	5.3	—
Fetches only	wipe	yes	6	13	no	10.3	—
Fetches only	sidecar	yes	6	13	no	8.4	—
Eight repeats	off	yes	8	0	no	2.1	—
Eight repeats	wipe	yes	8	17	no	6.3	—
Eight repeats	sidecar	yes	8	17	no	6.7	—
S3 runaway	off	yes	39	0	no	8.3	—
S3 runaway	wipe	no	10	22	yes	7.9	recursion cap
S3 runaway	sidecar	no	10	22	yes	8.1	recursion cap
S4 storm	off	yes	49	0	no	30.8	—
S4 storm	wipe	no	0	2	yes	1.5	burst entropy 0.96
S4 storm	sidecar	no	0	2	yes	1.9	burst entropy 0.96
Stale probe	wipe	—	0	1	no	—	allow after wipe
Stale probe	sidecar	—	0	1	yes	—	recursion cap
Fresh S1	wipe	yes	1	3	no	—	reports depth 0
Fresh S1	sidecar	yes	1	3	no	—	reports depth 0

6.4 Cross-request (not identity persistence)

Same sidecar. Isolate with S3, then:

Arm	Isolated	S1 done	Leftover depth	Probes	Reason
Probe without rewrite	yes	—	11	1	recursion cap
Fresh S1 reporting from 0	no	yes	0	3	—

This shows: the kernel keeps the last report until the next report overwrites it. It does not show an identity-bound isolate or dwell / recovery semantics.

Source: `persist_trace.csv` (3 October 2026). The same two probes also appear at the bottom of the 7 October `baseline_trace.csv` three-way run.

6.5 Two local planners

Source: `two_agent_trace.csv` (3 October 2026).

Layout	Agent	Job	Isolated	Done	Tools	Probes
Split sidecars	A	S3	yes	no	10	22
Split sidecars	B	S1	no	yes	1	3
Shared sidecar	A	S3	yes	no	10	22
Shared sidecar	B	S1	no	yes	1	3

Isolation is the **host monitor**, not a DID. On the shared sidecar, B's fresh report overwrites A's leftover depth, so B still completes. If two planners must not affect each other, give each a sidecar, or change the kernel to account per identity first.

6.6 Local small model (supplementary)

Ollama, Qwen2.5 0.5B, same graph and tools. Source: `llm_tool_trace_live.csv` (3 October 2026). Local inference. Scripted primary tables use zero hosted tokens. Live wall-clock is model-bound and is not an overhead figure.

Job	Gate	Done	Tools	Isolated	Prompt tokens	Reason
S1 short	off	yes	1	no	604	—
S1 short	on	yes	1	no	604	—
S2 long	off	no	0	no	18,859	LangGraph `recursion_limit` = 30
S2 long	on	no	10	yes	10,799	recursion cap at depth 11

0.5B does not complete the eight-step recipe. The gate stops the model's own runaway. That is not a false isolate of a finished long job. It is not LLM-agent validation.

6.7 Appendix: 500-node Monte Carlo (not a headline)

Death is defined as load (\ge 1). The simulated gate refuses any increment with `load + request > 0.95`. Under that pair of rules a node cannot die: **100% survival is an invariant of the construction**, not evidence about language-model agents.

Ungated seeds collapse: mean **1.876 / 500** alive at (T=100) over 1,000 paired runs (`survival_curve.csv`, 16 August 2026). Ablations locate the invariant in the breaker, not in a viability score (`ablation.csv`).

Do not use 100% as a title result.

Mean survival (%) at (T=100):

Malicious %	Full	No (v) floor	No breaker	No burst tax	Breaker only	(v) only / base
5	100	100	4.8	100	100	0.3 / 0.3
20	100	100	15.7	100	100	0.2 / 0.2

Malicious-rate sweep (`sensitivity_malicious.csv`, 300 runs). Every full-gate cell is the same invariant:

Mode	1%	5%	10%	15%	20%	30%	40%	50%
Baseline	2.3	0.3	0.3	0.3	0.2	0.2	0.2	0.2
Full	100	100	100	100	100	100	100	100

Reduced malicious per-request load (`adaptive_load.csv`, 5% malicious, 300 runs):

Malicious load	0.80	0.45	0.20	0.12
Baseline	0.3	0.6	3.1	7.8
Full	100	100	100	100

---

7. Evidence matrix and reproduction

7.1 What actually holds

Experiment	What holds	Strength
Bounded short / long / lookup / fetch / repeat	Gate does not block the tested live work	Strong inside those jobs

Experiment	What holds	Strength
Scripted infinite loop	Stopped at step 11	Strong inside that job
48-burst	Refused before GETs	Strong inside that job
Qwen 0.5B	Gate can sit on a local small model's tool calls	**Supplementary**
Leftover depth	Unrewritten probe still refused	**Telemetry retention**
New job reports 0	Not an irreversible identity lock	**Important bound**
Two sidecars	Planner state can be separated	Supports split-gate ops
Shared sidecar	Later planner can overwrite state	**Known limit**
Off / wipe / current	S1–S4 gated columns tie; stale probe splits	Comparative result holds
500-node mesh	Synthetic sanity check	Appendix only

7.2 How to reproduce

From the reference tree, with the sidecar spawned by each script:

```
PYTHONPATH=sdk/python python sdk/python/examples/runtime_trace.py --out
artifacts/neurips-aiwild-2026/results
PYTHONPATH=sdk/python python sdk/python/examples/llm_tool_trace.py --out
artifacts/neurips-aiwild-2026/results
PYTHONPATH=sdk/python python sdk/python/examples/baseline_trace.py --out
artifacts/neurips-aiwild-2026/results
PYTHONPATH=sdk/python python sdk/python/examples/persist_trace.py --out
artifacts/neurips-aiwild-2026/results
PYTHONPATH=sdk/python python sdk/python/examples/two_agent_trace.py --out
artifacts/neurips-aiwild-2026/results
```

Live local model (optional; requires a local OpenAI-compatible server):

```
OPENAI_BASE_URL=http://127.0.0.1:11434/v1
OPENAI_MODEL=qwen2.5:0.5b
PYTHONPATH=sdk/python python sdk/python/examples/llm_tool_trace.py --backend
live --out artifacts/neurips-aiwild-2026/results
```

Tests: `PYTHONPATH=sdk/python python -m pytest sdk/python/tests -q`

7.3 Reference constants

Name	Value
Recursion cap	10
Tool concurrency cap (C)	50
Memory / OOM ratio (r_{\mathrm{oom}})	0.90
Max context bytes (B)	(512\,\mathrm{MiB})
Pressure breaker (e^{\star})	0.95
Safe / warn bands	0.40 / 0.75
Delay (\delta(e))	(500\,\mathrm{ms}) at warn; (2000\,\mathrm{ms}) at (e=1)
Probation probe window	(1000\,\mathrm{ms})
S3 graph cap	40
S4 burst hint	48
Spec: isolation / probation dwell	64 / 128 epochs (**not validated on these traces**)
Spec: recover-score floor / re-entry	(0.3) / (0.8) (**not validated**)
Spec: Formula A (\alpha,\beta)	(0.95), (0.05) (**idle on these traces**)
Spec: Formula B (\lambda)	(0.01) / epoch (**idle**)
Spec: Formula C (\kappa)	(0.02) (**idle**)

---

8. Related work

Admission control and circuit breaking. Refusing work before overload is standard operations practice: TCP congestion control (Jacobson, SIGCOMM 1988), staged event-driven backpressure (Welsh, SEDA), the circuit-breaker pattern (Nygard, *Release It!*; Fowler), and production proxies such as Hystrix and Envoy. AFP-Enterprise does not invent that primitive. It attaches a host kernel — a depth cap, four-axis pressure, retained telemetry, and a specified recover path — **before a planner–tool hop**, and says which of those stages the traces actually reach. What this report measures is leftover host telemetry, not a lock on agent identity.

Agent frameworks (ReAct, AutoGen, MetaGPT, CAMEL, LangGraph, CrewAI, generative agents, Reflexion) optimize task completion. They are not a host kernel at the execution boundary.

Agent safety evaluations (AgentDojo, AgentHarm, injection studies) grade speech and tool policy. They do not ask whether a well-formed graph can still saturate its host.

Control and oversight (AI control, debate, constitutional feedback, scalable oversight, representation-level circuit breakers) constrain *intent*. AFP-Enterprise constrains *capacity at the boundary*. A monitored but unbraked fleet can still loop itself into unavailability.

Agent protocols (MCP, A2A) secure tools and sessions. A valid MCP call can still be a physically illegal next hop. The layers compose: policy and injection defense first; this kernel second.

---

9. What this report does not claim

- Production, in-the-wild, computer-use, or MCP-ingress evaluation
- Identity-persistent penalties or identity-rotation defense
- That the delay band or isolate–recover machine was exercised
- That 500-node 100% survival is a field result
- That Qwen 0.5B validates a long-horizon LLM planner
- Per-identity isolation when two planners share one sidecar
- That AFP-Enterprise replaces rate limits, sandboxes, or human oversight
- Scalability, throughput, or general prevention of resource exhaustion

This report does not prove that a planner–tool fleet is safe.

---

10. Closing

Chapter 1 asked who governs the optimizer before it optimizes. AFP-Enterprise answers at the local execution boundary: a host kernel, reached by a pre-flight probe, then the tool runs.

The empirical claim is narrower than older mixed drafts, and cleaner for it:

> On the jobs that were tested, the depth cap and the hard pressure isolate fire. S1–S4 do not distinguish a per-step wipe from the current sidecar. The difference appears only when a later execution reuses stale, uncleared telemetry. The delay band and the isolate–recover machine are specified; they are not evidenced by these tables.

That locates AFP-Enterprise's extra measured behavior. It does not license a persistence slogan, a mesh survival percentage, or a production-safety badge.

Legal notice

© 2026 Muchen He. This document is licensed under CC BY-NC-SA 4.0. The author is an independent researcher and lists no institutional affiliation. Shared use of the word "Aegis" with other academic projects must not be read as shared architecture or shared lineage.

Reference software, if distributed separately, is typically MIT-licensed and experimentally unaudited. This manuscript is a technical report, not a conference submission, and is not anonymous.

References (selected)

- Jacobson, V. Congestion avoidance and control. *SIGCOMM*, 1988.
- Welsh, M., Culler, D., and Brewer, E. SEDA: An architecture for well-conditioned, scalable Internet services. *SOSP*, 2001.
- Nygard, M. *Release It!* Pragmatic Bookshelf, 2018 (2nd ed.).
- Fowler, M. CircuitBreaker. martinfowler.com, 2014.
- Netflix. Hystrix. 2012.
- Envoy Proxy. Circuit breaking. Documentation, 2024.
- Yao, S., et al. ReAct. *ICLR*, 2023.
- Wu, Q., et al. AutoGen. 2023.
- LangChain. LangGraph. 2025.
- Anthropic. Model Context Protocol. 2024.
- Google. Agent2Agent protocol. 2025.
- Greshake, K., et al. Not what you've signed up for. 2023.
- Debenedetti, E., et al. AgentDojo. 2024.
- Andriushchenko, M., et al. AgentHarm. 2024.
- Greenblatt, R., et al. AI control. 2024.
- Cemri, M., et al. Why do multi-agent LLM systems fail? 2025.
- He, M. Aegis Fabric Protocol v1 empirical baseline. Zenodo, 2026. DOI 10.5281/zenodo.20674352.

Citation

```
He, Muchen (2026). AFP–Enterprise: Host Runtime Kernel for
Planner–Tool Stacks. Zenodo. [assign DOI on upload]
```

Suggested BibTeX after the Zenodo DOI is reserved:

```
@techreport{he2026afpenterprise,
  author      = {He, Muchen},
  title       = {{AFP-Enterprise}: Host Runtime Kernel for
                 Planner--Tool Stacks},
  year        = {2026},
  month       = oct,
  institution = {Zenodo},
  type        = {Technical report},
  doi         = {TO-BE-ASSIGNED}
}
```

Revision

Date	Note
2026-10-07	First AFP-Enterprise English archival draft. Claims and tables from laptop planner–tool traces. Increment stated as stale-state retention.
2026-10-07	Full numeric tables from the current result CSVs: depth-only, S1–S4, latest three-way `baseline_trace` (7 Oct 11:27), persist leftover depth, two-planner tools/probes, live tokens and LangGraph `recursion_limit` = 30, mesh ablation plus rate and adaptive-load sweeps (ungated endpoint 1.876/500). Claims unchanged.
2026-10-07	Interface + kernel: pre-flight is the call; four-stage host kernel is specified; traces still only evidence the depth cap, hard isolate, and leftover telemetry.
2026-10-07	Restored concrete kernel math from the earlier archival draft: four pressure axes, delay (\delta(e)), isolate–recover transitions, Formulas A–C. Measured vs specified kept separate.